

제 2장 윈도우 기본 입출력 3

2026년 1학기 윈도우 프로그래밍

8. 직선, 원, 사각형, 다각형 그리기

- 점 그리기
- 직선, 원, 사각형, 다각형 그리기
- 도형 속성 변경하기

점 그리기

- 원도우에 점 그리기

COLORREF SetPixel (HDC hDC, int nXpos, int nYpos, COLORREF color);

- hDC: DC 핸들
- nXpos, nYpos: 설정할 점의 x, y 좌표
- color: 점을 그리는데 사용 할 색상

BOOL SetPixelV (HDC hDC, int nxPos, int nYpos, COLORREF color);

- hDC: DC 핸들
- nXpos, nYpos: 설정할 점의 x, y 좌표
- color: 점을 그리는데 사용 할 색상

- 지정한 좌표에서 픽셀의 색상값 가져오기

COLORREF GetPixel (HDC hDC, int nXPos, int nYpos);

- hDC: DC 핸들
- nXpos, nYpos: 설정할 점의 x, y 좌표
- 리턴 값: 픽셀의 색상

직선 그리기

- 직선의 시작점으로 이동하기 함수

```
BOOL MoveToEx (HDC hDC, int X1, int Y1, LPPPOINT lpPoint );
```

- hDC: DC 핸들
- X1, Y1: 직선의 시작점 (x와 y 좌표값)
- lpPoint: 이전의 좌표, 사용 안함

(X1, Y1)

(X2, Y2)

- 직선의 시작점에서 끝점까지 직선 그리기 함수

```
BOOL LineTo (HDC hDC, int X2, int Y2 );
```

- hDC: DC 핸들
- X2, Y2: 직선의 끝점

```
case WM_PAINT :
```

```
    hDC = BeginPaint (hWnd, &ps) ;
```

```
    MoveToEx (hDC, 10, 30, NULL);
```

```
    LineTo (hDC, 50, 60);           //-- (10, 30) → (50, 60) 직선 그리기
```

```
    EndPaint (hWnd, &ps) ;
```

```
    return 0 ;
```

(10, 30)

(50, 60)

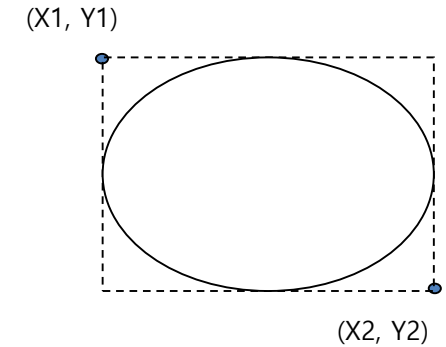
```
typedef struct tagPOINT {  
    LONG x;  
    LONG y;  
} POINT;
```

원 그리기

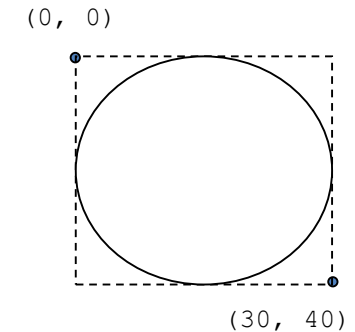
- 두 점의 좌표를 기준으로 만들어진 가상의 사각형에 내접하는 원을 그리는 함수

```
BOOL Ellipse (HDC hDC, int X1, int Y1, int X2, int Y2 );
```

- hDC: DC 핸들
- X1, Y1: 좌측 상단 좌표값 (x와 y 최소값)
- X2, Y2: 우측 하단 좌표값 (x와 y 최대값)



```
case WM_PAINT :  
    hDC = BeginPaint (hWnd, &ps) ;  
    Ellipse(hDC, 0, 0, 30, 40);      //-- 박스의 중심점 (15,20)  
    EndPaint (hWnd, &ps) ;  
    return 0 ;
```

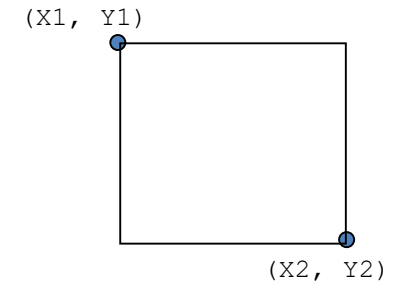


사각형 그리기

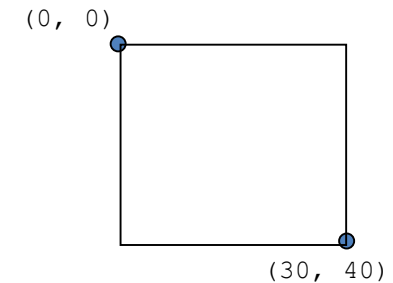
- 두 점의 좌표를 기준으로 수평수직 사각형을 그림

```
BOOL Rectangle (HDC hdc, int X1, int Y1, int X2, int Y2 );
```

- hdc: DC 핸들
- X1, Y1: 좌측 상단 좌표값 (x와 y 최소값)
- X2, Y2: 우측 하단 좌표값 (x와 y 최대값)



```
case WM_PAINT :  
    hdc = BeginPaint (hWnd, &ps) ;  
    Rectangle(hdc, 0, 0, 30, 40);    //-- 원을 둘러싼 사각형의 좌표  
    EndPaint (hWnd, &ps) ;  
    return 0 ;
```



사각형 그리기

- 사각형 그리기 다른 함수들
 - 내부가 채워진 사각형 그리기:

```
int FillRect (HDC hDC, CONST RECT *lprc, HBRUSH hbr)
```

- hDC: DC 핸들
- lprc: 사각형의 좌표값
- hbr: 내부 색



- 외곽선 사각형 그리기:

```
int FrameRect (HDC hDC, CONST RECT *lprc, HBRUSH hbr);
```

- hDC: DC 핸들
- lprc: 사각형의 좌표값
- hbr: 외곽선 색



- 반전 사각형 그리기:

```
int InvertRect (HDC hDC, CONST RECT *lprc);
```

- hDC: DC 핸들
- lprc: 사각형의 좌표값



- 그리기 이전의 픽셀의 색과 반전된 사각형 그리기

```
typedef struct _RECT {  
    LONG left;  
    LONG top;  
    LONG right;  
    LONG bottom;  
} RECT;
```

실제 사용할 때:

RECT rect = {left좌표, top좌표, right좌표, bottom좌표};

FrameRect (hDC, &rect, hBrush);
InvertRect (hDC, &rect);

사각형 그리기

- 몇가지 알아두면 편리한 사각형 관련 함수들

BOOL OffsetRect (LPRECT lprc, int dx, int dy);

- 주어진 Rect를 dx, dy만큼 이동한다.

BOOL InflateRect (LPRECT lprc, int dx, int dy);

- 주어진 Rect를 dx, dy만큼 늘이거나 줄인다.

BOOL IntersectRect (LPRECT lprcDest, CONST RECT *lprcSrc1, CONST RECT lprcSrc2);

- 두 RECT (lprcSrc1, lprcSrc2)가 교차되었는지 검사한다.
- lprcDest: 교차된 RECT 부분의 좌표값이 저장된다.

BOOL UnionRect (LPRECT lprcDest, CONST RECT *lprcSrc1, CONST RECT *lprcSrc2)

- 두 RECT (lprcSrc1, lprcSrc2) 를 union 시킨다.
- lprcDest: 두 사각형을 합한 사각형의 좌표값이 저장된다.

BOOL PtInRect (CONST RECT *lprc, POINT pt);

- 특정 좌표 pt가 lprc 영역 안에 있는지 검사한다.

BOOL SetRect (LPRECT lprc, int xLeft, int yTop, int xRight, int yBottom);

- RECT 구조체의 좌표를 직접 설정한다.

BOOL EqualRect (const RECT *lprc1, const RECT *lprc2);

- 두 사각형이 동일한지 비교한다.

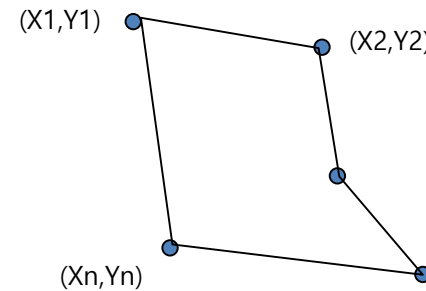
다각형 그리기

- 연속되는 여러 점의 좌표를 직선으로 연결하여 다각형을 그림

BOOL Polygon (HDC hDC, CONST POINT *lppt, int cPoints);

- hDC: DC 핸들
- lppt: 다각형 꼭지점의 좌표 값이 저장된 리스트
- cPoints: 꼭지점의 개수

```
typedef struct tagPOINT {  
    LONG x;  
    LONG y;  
} POINT;
```

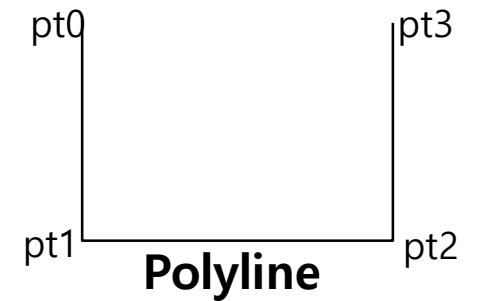
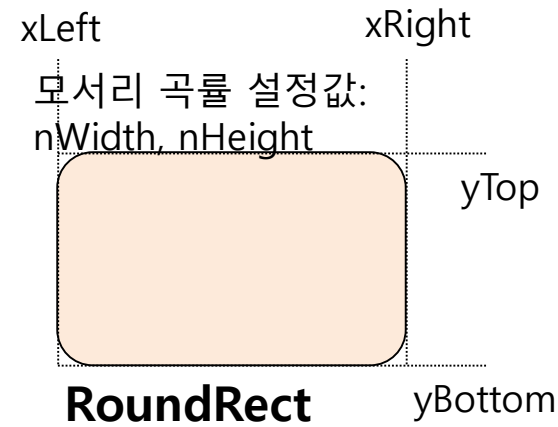
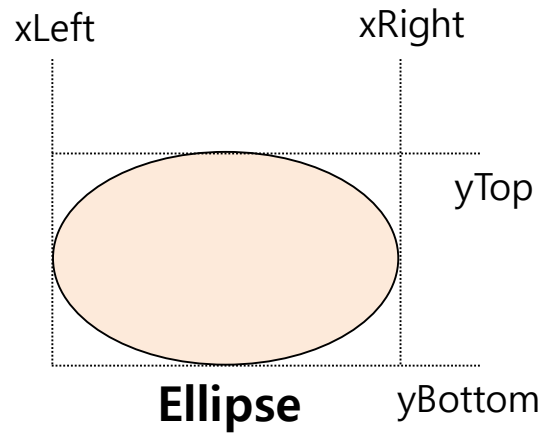
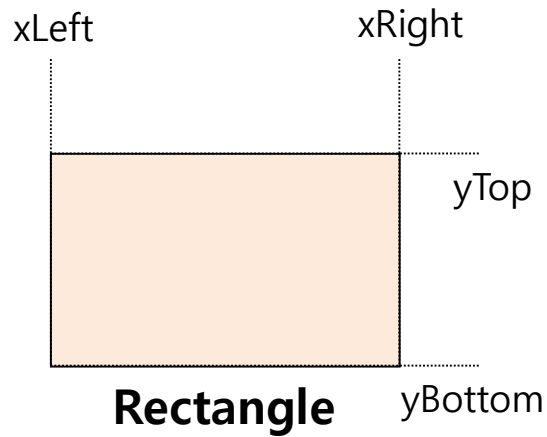


```
POINT point[10] = {{10,20}, {100,30}, {500,200}, {600, 300}, {200, 300}};
```

```
case WM_PAINT :  
    hDC = BeginPaint (hWnd, &ps) ;  
    Polygon(hDC, point, 5);    // 5각형  
    EndPaint (hWnd, &ps) ;  
    return 0 ;
```

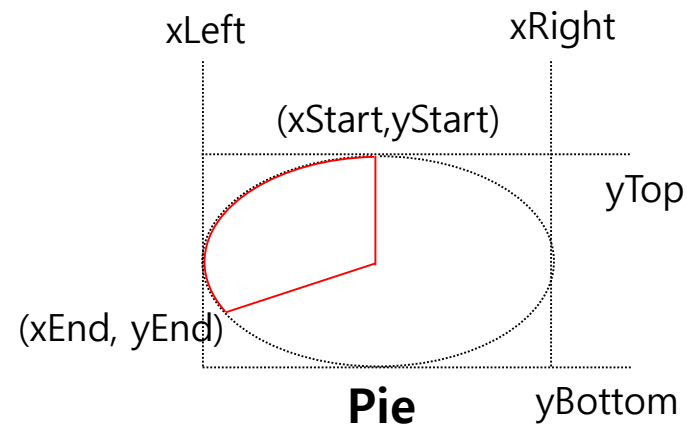
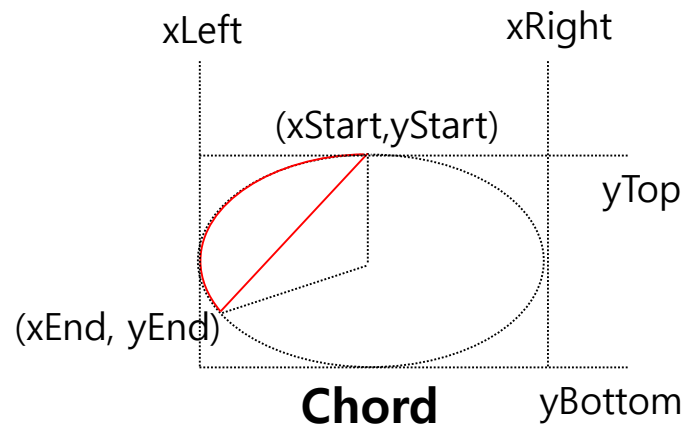
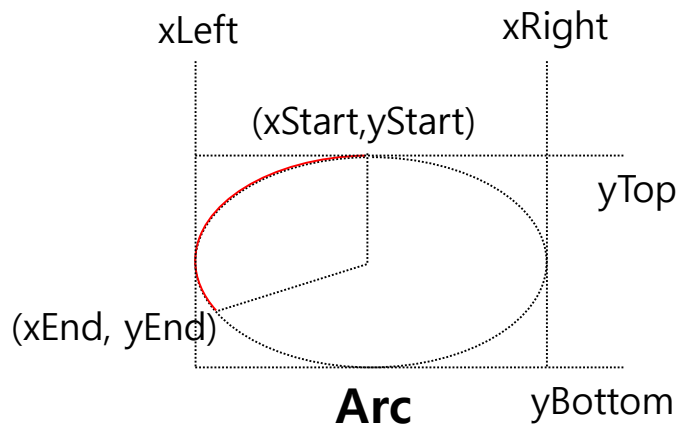
도형 그리기

- 다양한 도형 그리기
 - `BOOL Rectangle` (HDC hdc, int xLeft, int yTop, int xRight, int yBottom);
 - `BOOL Ellipse` (HDC hdc, int X1, int Y1, int X2, int Y2);
 - `BOOL RoundRect` (HDC hdc, int xLeft, int yTop, int xRight, int yBottom, int nWidth, int nHeight);
 - `BOOL Polyline` (HDC hdc, CONST POINT *lppt, int cPoints);

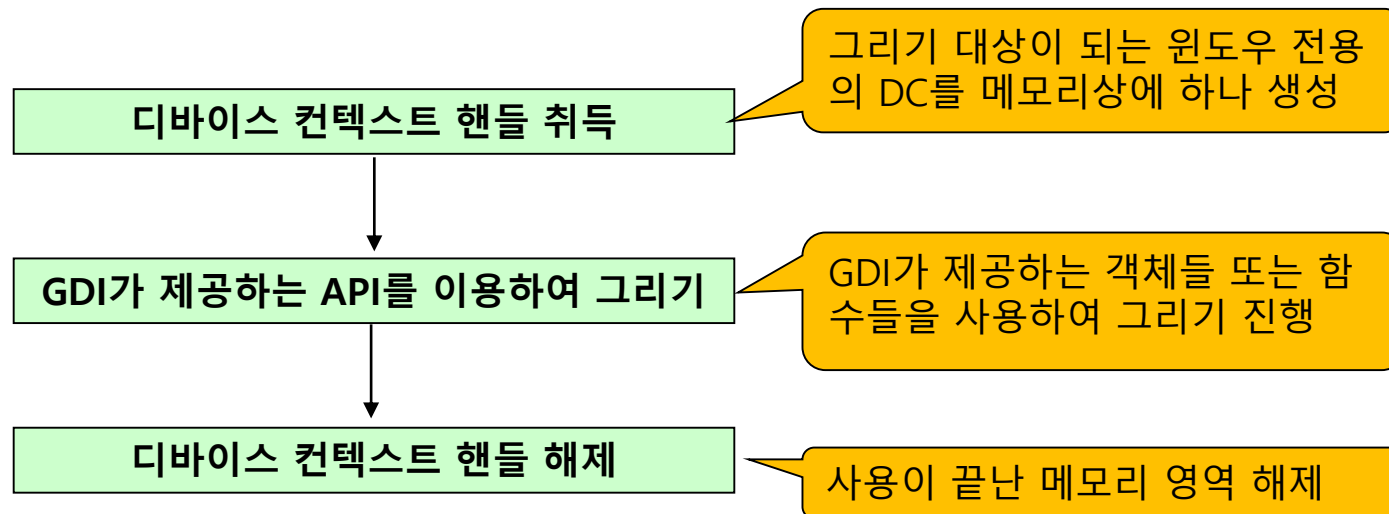


도형 그리기

- 다양한 도형 그리기
 - `BOOL Arc (HDC hDC, int xLeft, int yTop, int xRight, int yBottom, int xStart, int yStart, int xEnd, int yEnd );`
 - `BOOL Chord (HDC hDC, int xLeft, int yTop, int xRight, int yBottom, int xStart, int yStart, int xEnd, int yEnd );`
 - `BOOL Pie (HDC hDC, int xLeft, int yTop, int xRight, int yBottom, int xStart, int yStart, int xEnd, int yEnd );`
- 기본 그리기 방향은 시계 반대방향



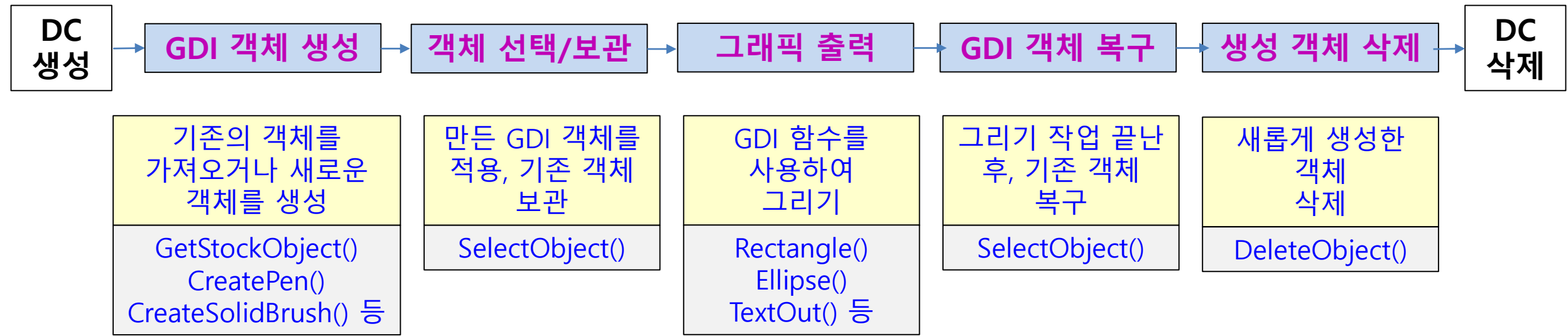
- **GDI (Graphic Device Interface)**
 - 화면, 프린터 등의 모든 출력장치를 제어하는 윈도우 모듈
 - GDI 오브젝트: 그림을 그리는 데 필요한 도구 (펜, 브러쉬 등)
 - GDI를 사용한 그리기 순서



- 윈도우에서 기본적으로 제공하는 GDI객체인 스톡 객체는 따로 생성할 필요가 없어서 편리하지만 다양하게 그래픽을 하기에는 부족
 - 다양한 출력을 위해서는 GDI객체를 만들어서 사용
 - GDI객체를 만드는 함수는 앞의 "Create"로 시작하는 함수

GDI 오브젝트	의미	내용	핸들타입	디폴트 값
펜	선을 그을 때	• 선을 그리거나 영역의 경계선을 그릴 때 사용 • 선의 색, 두께, 형태 등을 지정 • 디폴트는 검은색 1픽셀 실선	HPEN	검정색의 가는 실선
브러시	면을 채울 때	• 어떤 영역의 내부를 채울 때 사용 • 채우기 색, 채우기 패턴 등을 지정 • 디폴트는 흰색	HBRUSH	흰색
폰트	문자 출력 글꼴	• 문자를 출력할 때 사용하며 색, 모양, 크기 등을 지정 • 디폴트는 시스템 폰트	HFONT	시스템 글꼴
비트맵	비트맵 이미지	• 비트맵 그림 파일에 관한 옵션	HBITMAP	-
팔레트	팔레트	• 화면에 출력할 수 있는 색에 제한을 받을 경우, 실제로 화면에 출력할 색의 수 등을 지정	HPALETTE	-
리전	화면상의 영역	• 임의의 도형을 그리는 것과 관련된 옵션을 설정	HRGN	-

GDI 객체를 사용하는 방법



- GDI객체를 만드는 함수
 - 펜:
 - `HPEN CreatePen ( int fnPenStyle, int nWidth, COLORREF crColor);`
 - 브러시:
 - `HBRUSH CreateSolidBrush (COLORREF crColor);`
 - `HBRUSH CreateHatchBrush (int iHatch, COLORREF crColor);`
 - 폰트(글꼴):
 - `HFONT CreateFont (int cHeight, int cWidth, int cEscapement, int cOrientation, int cWeight, DWORD bItalic, DWORD bUnderline, DWORD bStrikeOut, DWORD iCharSet, DWORD iOutPrecision, DWORD iClipPrecision, DWORD iQuality, DWORD iPitchAndFamily, LPCSTR pszFaceName );`
 - `HFONT CreateFontIndirect (const LOGFONT *lpLf);`
 - 비트맵:
 - `HBITMAP CreateBitmap (int nWidth, int nHeight, UINT nPlanes, UINT nBitCount, const VOID *lpBits );`
 - `HBITMAP CreateCompatibleBitmap (HDC hDC, int cx, int cy );`

펜 : 선 다루기

- 선의 굵기 정보와 색상정보를 가지는 펜 핸들을 생성 함수

HPEN CreatePen (int fnPenStyle, int nWidth, COLORREF crColor);

- fnPenStyle: 펜 스타일
- nWidth: 펜의 굵기로 단위는 픽셀
- crColor: 색상을 표현하기 위해 COLORREF 값을 제공하며, RGB()함수로 만들

PS_SOLID	
PS_DASH	
PS_DOT	
PS_DASHDOT	
PS_DASHDOTDOT	

- 그림을 그릴 화면인 DC에 펜 핸들들을 등록함수

HGDIOBJ **SelectObject** (HDC hDC, HGDIOBJ hgdiobj);

HPEN **SelectObject** (HDC hDC, HPEN pen);

- 그림 그리기를 마친 후 생성된 펜 핸들은 삭제 함수

BOOL **DeleteObject** (HGDI OBJ hgdiob);

BOOL DeleteObject (HPEN pen);

- **COLORREF RGB (int Red, int Green, int Blue)**

- Red, Green, Blue에는 0~255 사이의 정수 값을 사용
- COLORREF 는 색상을 표현하는 자료형 (R, G, B 3가지 색으로 표현)

빨간 점선으로 원 그리기

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
```

```
{
```

```
    HDC hDC;
```

```
    HPEN hPen, oldPen;
```

```
    switch (iMsg) {
```

```
        case WM_PAINT :
```

```
            hDC = BeginPaint (hWnd, &ps) ;
```

```
            // DC 얻어오기
```

```
            hPen = CreatePen (PS_DOT, 1, RGB(255,0,0));
```

```
            // GDI: 펜 만들기
```

```
            oldPen = (HPEN) SelectObject (hDC, hPen);
```

```
            // 새로운 펜 선택하기
```

```
            Ellipse(hDC, 20,20, 300,300);
```

```
            // 선택한 펜으로 도형 그리기
```

```
            SelectObject (hDC, oldPen);
```

```
            // 이전의 펜으로 돌아감
```

```
            DeleteObject (hPen);
```

```
            // 만든 펜 객체 삭제하기
```

```
            EndPaint (hWnd, &ps) ;
```

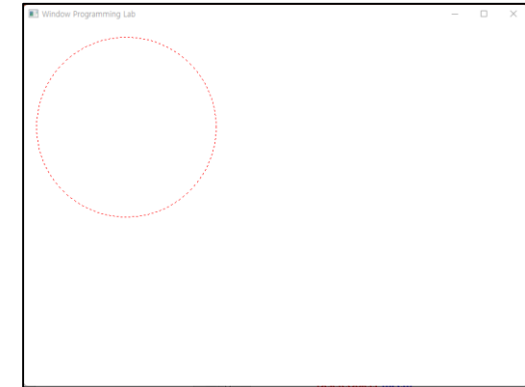
```
            // DC 해제하기
```

```
            break;
```

```
    }
```

```
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
```

```
}
```



면 색상 변경

- 면의 색상정보를 가지는 브러시 핸들을 만들어 주는 함수

```
HBRUSH CreateSolidBrush (COLORREF crColor);
```

– crColor: 면의 색상 값

- 그림을 그릴 화면인 디바이스 컨텍스트에 브러시 핸들을 등록

```
HGDIOBJ SelectObject (HDC hDC, HGDIOBJ hgdiobj);
```

→

```
HBRUSH SelectObject (HDC hDC, HBRUSH hBrush);
```

- 그림 그리기를 마친 후 생성된 브러시 핸들은 삭제

```
BOOL DeleteObject (HGDIOBJ hgdiobj);
```

→

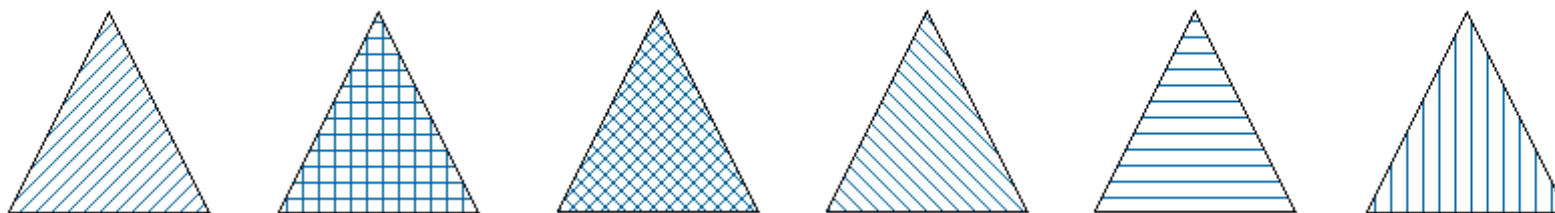
```
BOOL DeleteObject (HBRUSH hBrush);
```

면 색상 변경 (빗살무늬)

- 면에 빗살무늬와 색상정보를 가지는 브러시 핸들을 만들어 주는 함수

HBRUSH CreateHatchBrush (int iHatch, COLORREF crColor);

- iHatch: 브러시 해치 스타일
 - HS_BDIAGONAL: 45도 위쪽 왼쪽에서 오른쪽 해치
 - HS_CROSS: 가로 및 세로 크로스패치
 - HS_DIAGCROSS: 45도 크로스해치
 - HS_FDIAGONAL: 왼쪽에서 오른쪽으로 45도 아래쪽 해치
 - HS_HORIZONTAL: 가로 빗살 무늬
 - HS_VERTICAL: 세로 해치
- crColor: 면의 색상 값



빨간면의 원 그리기

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
```

```
{
```

```
    HDC hDC;
```

```
    HBRUSH hBrush, oldBrush;
```

```
    switch (iMsg) {
```

```
        case WM_PAINT :
```

```
            hDC = BeginPaint (hWnd, &ps) ;
```

```
            // DC 얻어오기
```

```
            hBrush = CreateSolidBrush (RGB(255,0,0));
```

```
            // GDI: 브러시 만들기
```

```
            oldBrush = (HBRUSH) SelectObject (hDC, hBrush);
```

```
            // 새로운 브러시 선택하기
```

```
            Ellipse (hDC, 20,20, 300,300);
```

```
            // 선택한 브러시로 도형 그리기
```

```
            SelectObject (hDC, oldBrush);
```

```
            // 이전의 브러시로 돌아가기
```

```
            DeleteObject (hBrush);
```

```
            // 만든 브러시 객체 삭제하기
```

```
            EndPaint (hWnd, &ps) ;
```

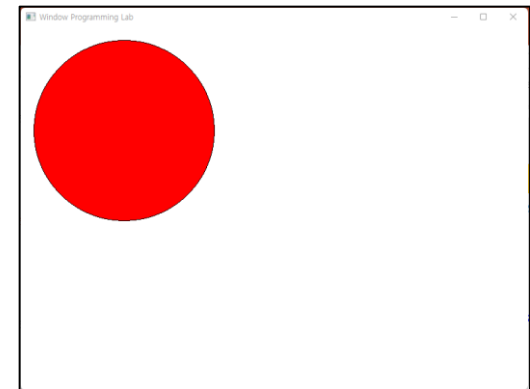
```
            // DC 해제하기
```

```
            return 0 ;
```

```
    }
```

```
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
```

```
}
```



GDI 객체 핸들 구하기 함수들

- 생성한 펜, 브러시 및 폰트를 설정하는 함수
 - 지정된 디바이스 컨텍스트로 객체를 선택한다.
 - 새로운 객체는 동일한 형식의 이전 객체를 대체한다.

HGDIOBJ SelectObject (HDC hDC, HGDIOBJ hgdiobj);

- hDC: DC 핸들값
- hgdiobj: GDI의 객체
- 리턴 값은 원래의 오브젝트 값

- 생성한 객체 삭제 함수

BOOL DeleteObject (HGDIOBJ hgdiobj);

- GDI오브젝트를 삭제하고 오브젝트가 사용하던 시스템 리소스를 해제
- 객체 생성 함수로 만들어진 GDI 오브젝트는 반드시 삭제해주어야 한다.
- hgdiobj: GDI의 객체

기존의 GDI 객체 사용하기

- 윈도우가 제공하는 펜 브러시 및 폰트를 취득하는 함수
 - 스톡 오브젝트: 윈도우에서 기본적으로 제공하는 GDI 객체
 - 윈도우가 제공해주므로 따로 생성하지 않고 사용할 수 있으며 해제하지 않아도 됨.

HGDIOBJ GetStockObject (int fnObject);

- 리턴 값은 가져올 스톡 오브젝트 핸들 값
- fnObject: 가져올 스톡 오브젝트의 속성
 - 브러시 속성: BLACK_BRUSH / DKGRAY_BRUSH / DC_BRUSH / GRAY_BRUSH / HOLLOW_BRUSH / LTGRAY_BRUSH / NULL_BRUSH / WHITE_BRUSH
 - DC_BRUSH: 단색 브러시, 기본색은 흰색
 - HOLLOW_BRUSH: 속이 빈 브러시 (NULL_BRUSH)
 - 펜 속성: BLACK_PEN / DC_PEN / WHITE_PEN / NULL_PEN
 - DC_PEN: 단색 펜, 기본색은 검정색
 - NULL_PEN: 아무것도 그리지 않는다.

- 클라이언트의 영역을 취득하는 함수

BOOL GetClientRect (HWND hWnd, LPRECT lprc);

- 클라이언트의 영역을 취득한다.
- hWnd: 윈도우 핸들
- lprc: 윈도우 영역의 좌표값

GDI 객체 핸들하기

- 검정색 테두리를 가지고 회색 내부를 가진 사각형을 그리는 경우
 - 펜은 변경하지 않고, 브러시 색상만 변경
 - 객체를 만들지 않고 윈도우가 제공하는 브러시 객체 (스톡 객체) 사용

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
```

```
{  
    HDC hDC;  
    HBRUSH hBrush, oldBrush;  
  
    switch (iMsg) {  
        case WM_PAINT :  
            hDC = BeginPaint (hWnd, &ps) ;  
                                                    // DC 얻어오기  
  
            hBrush = (HBRUSH) GetStockObject (GRAY_BRUSH);  
            oldBrush = (HBRUSH) SelectObject (hDC, hBrush);  
                                                    // 윈도우가 제공하는 객체 가져오기  
  
            Rectangle (hDC, 50, 50, 300, 200);  
  
            SelectObject (hDC, oldBrush);  
                                                    // 제자리 돌아가기  
  
            EndPaint (hWnd, &ps) ;  
            return 0 ;  
                                                    // DC 해제하기  
        }  
    }  
    return DefWindowProc (hWnd, iMsg, wParam, lParam);  
}
```



GDI 객체 핸들하기

- 파란색 테두리를 가지고, 노란색 내부를 가진 사각형을 그리는 경우
→ 파란색의 펜 객체와 노란색 브러시 객체를 만들어 사용

```
LRESULT CALLBACK WndProc (HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
```

```
{
    HDC hDC;
    HPEN hPen, oldPen;
    HBRUSH hBrush, oldBrush;

    switch (iMsg) {
        case WM_PAINT :
            hDC = BeginPaint (hWnd, &ps) ;                                // DC 얻어오기

            hPen = CreatePen (PS_SOLID, 5, RGB(0, 0, 255));                // 새로운 객체 만들기: 펜
            oldPen = (HPEN) SelectObject (hDC, hPen);
            hBrush = CreateSolidBrush (RGB(255, 255, 0));                  // 새로운 객체 만들기: 브러쉬
            oldBrush = (HBRUSH) SelectObject (hDC, hBrush);

            Rectangle (hDC, 50, 50, 300, 200);

            SelectObject (hDC, oldPen);                                    // 제자리 돌아가기
            SelectObject (hDC, oldBrush);                                // 새로운 객체 삭제
            DeleteObject (hPen);
            DeleteObject (hBrush);

            EndPaint (hWnd, &ps) ;                                        // DC 해제하기
            return 0 ;
    }
    return DefWindowProc (hWnd, iMsg, wParam, lParam);
}
```

